

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of
Science and Commerce Department of Information
Technology (B.Sc.I.T Semester V)

AI

Task

Roll No.:T023	Name:som Mishra
Class: TYIT	Batch:1
Date of Assignment:29-07-2026	Date/Time of Submission:29-07-2026

1. Design PEAS description for an Online Food Delivery Agent and classify its environment.

Code:-

```
# PEAS Description

performance = [
    "Fast Delivery","Correct Order","Customer Satisfaction","Low Delivery
Cost"
]

environment = [
    "Customer","Restaurant", "Roads","Traffic","Weather"
]

actuators = [
    "Accept Order","Assign Delivery Boy","Send Notification","Update Order
Status"
]

sensors = [
    "GPS", "Customer Location", "Restaurant Status","Traffic Data"
```

```
]

print("PEAS Description")

print("\nPerformance Measure:")
for i in performance:
    print("-", i)

print("\nEnvironment:")
for i in environment:
    print("-", i)

print("\nActuators:")
for i in actuators:
    print("-", i)

print("\nSensors:")
for i in sensors:
    print("-", i)

print("\nEnvironment Classification")
print("Partially Observable")
print("Stochastic")
print("Sequential")
print("Dynamic")
print("Continuous")
print("Multi-Agent")
```

Output:-

PEAS Description

Performance Measure:

- Fast Delivery
- Correct Order
- Customer Satisfaction
- Low Delivery Cost

Environment:

- Customer
- Restaurant
- Roads
- Traffic
- Weather

Actuators:

- Accept Order
- Assign Delivery Boy
- Send Notification
- Update Order Status

Sensors:

- GPS
- Customer Location
- Restaurant Status
- Traffic Data

Environment Classification

Partially Observable

Stochastic

Sequential

Dynamic

2. Implement Breadth-First Search to find the shortest path in a simple maze.

Code:-

```
from collections import deque

maze = [
    ['S',0,1],
    [0,0,0],
    [1,0,'G']
]

start = (0,0)
goal = (2,2)
```

```

queue = deque([(start,[start])])
visited = []

while queue:
    node,path = queue.popleft()

    if node==goal:
        print("Shortest Path")
        print(path)
        break

    if node not in visited:
        visited.append(node)

        x,y=node

        moves=[(x+1,y),(x-1,y),(x,y+1),(x,y-1)]

        for nx,ny in moves:

            if 0<=nx<3 and 0<=ny<3:

                if maze[nx][ny]!=1 and (nx,ny) not in visited:
                    queue.append((nx,ny),path+[(nx,ny)])

```

Output:-

```

Shortest Path
[(0, 0), (1, 0), (1, 1), (2, 1), (2, 2)]

```

3. Apply Bayes' Rule to calculate the probability of rain given cloudy weather.

Code:-

```

P_rain = 0.4
P_cloudy_given_rain = 0.8
P_cloudy = 0.5

P_rain_given_cloudy = (P_cloudy_given_rain * P_rain) / P_cloudy

print("Probability of Rain given Cloudy =", P_rain_given_cloudy)

```

Output:-

Probability of Rain given Cloudy = 0.6400000000000001

4. Solve the Missionaries and Cannibals problem using BFS or DFS.

Code:-

```
from collections import deque

start = (3,3,0)
goal = (0,0,1)

moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]

def safe(m,c):
    if m<0 or c<0 or m>3 or c>3:
        return False

    if m>0 and c>m:
        return False

    rm=3-m
    rc=3-c

    if rm>0 and rc>rm:
        return False

    return True

queue = deque([(start,[start])])
visited=[]

while queue:

    state,path = queue.popleft()

    if state==goal:
        print("Solution")
        for i in path:
            print(i)
        break

    if state not in visited:
```

```

        visited.append(state)

        m,c,b=state

        for dm,dc in moves:

            if b==0:
                new=(m-dm,c-dc,1)
            else:
                new=(m+dm,c+dc,0)

            if safe(new[0],new[1]):
                queue.append((new,path+[new]))

```

Output:-

```

Solution
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)

```

Code:-

```

import heapq

graph = {
    'A': [('B',1), ('C',4)],
    'B': [('D',2)],
    'C': [('E',1)],
    'D': [('G',3)],

```

```

'E': [('G', 2)],
'G': []
}

heuristic = {
'A': 5,
'B': 4,
'C': 2,
'D': 2,
'E': 1,
'G': 0
}

def astar(start, goal):

    pq = [(heuristic[start], 0, start, [start])]

    while pq:

        f, cost, node, path = heapq.heappop(pq)

        if node == goal:
            return path, cost

        for nxt, w in graph[node]:

            heapq.heappush(pq,
                (cost + w + heuristic[nxt], cost + w, nxt, path + [nxt]))

def ucs(start, goal):

    pq = [(0, start, [start])]

    while pq:

        cost, node, path = heapq.heappop(pq)

        if node == goal:
            return path, cost

```

```

        for nxt,w in graph[node]:

            heapq.heappush(pq,
                           (cost+w,nxt,path+[nxt]))

path1,cost1=astar('A','G')
path2,cost2=ucs('A','G')

print("A* Search")
print("Path =",path1)
print("Cost =",cost1)

print()

print("Uniform Cost Search")
print("Path =",path2)
print("Cost =",cost2)

```

Output:-

```

A* Search
Path = ['A', 'B', 'D', 'G']
Cost = 6

Uniform Cost Search
Path = ['A', 'B', 'D', 'G']
Cost = 6

```